JiT-Inspired Diffusion Model for CIFAR-10 Image Generation: A Reduced-Scale Implementation, Ablation Study, and Comparison with the Original Paper

Final Project Report

***Abstract*—**This report presents a compact, JiT-inspired diffusion denoiser for 32×32 CIFAR-10 images. A patch-based Transformer predicts the clean image $\hat{\mathbf{x}}$ from a linearly interpolated noisy input and is trained with a velocity-space mean-squared-error (v-loss) objective. Two patch sizes (4 and 8) are trained for 10 epochs, and a third model (Patch-4 without time embedding) serves as an ablation. The recorded test v-losses are 0.204914 (Patch-4), 0.201217 (Patch-8), and 0.208269 (Patch-4, no time embedding). Removing time embedding raises the Patch-4 test v-loss by 0.003355 (about 1.64%). All figures are taken directly from the project notebook. Every difference comes from one training run per configuration, so none is established as statistically reliable. The study is a reduced-scale educational implementation and is not a reproduction of the ImageNet-scale experiments in the original paper.

***Index Terms*—**diffusion models, image generation, Vision Transformer, clean-image prediction, denoising, CIFAR-10, ablation study.



Fig. 1. Ten CIFAR-10 training images and their class labels, as visualized in the notebook.

I. Introduction and Objectives

Diffusion models learn to reverse a gradual corruption process. The paper "Back to Basics: Let Denoising Generative Models Denoise" [1] argues that the network should predict the *clean* image directly instead of noise or a noised quantity such as velocity. It proposes JiT ("Just image Transformers"): plain, large-patch Transformers operating on raw pixels, with no tokenizer, no pre-training, and no extra loss.

This project is a small, runnable PyTorch implementation inspired by that idea. Its objectives are to (i) build a patch-based Transformer that predicts clean pixels, (ii) train it with the velocity-space loss used in the notebook, (iii) compare patch sizes 4 and 8, (iv) isolate the effect of time conditioning with an ablation, and (v) compare the setup honestly with the original paper. All numbers and figures come from the notebook `JiT_Project.ipynb` (Google Colab, Tesla T4 GPU, PyTorch 2.11).

II. Dataset and Preprocessing

CIFAR-10 [2] has 60,000 colour images of size 32×32 in 10 classes, split into 50,000 training and 10,000 test images. Images are converted to tensors and normalized per channel with mean 0.5 and standard deviation 0.5, which maps pixels to $[-1, 1]$. No data augmentation is used. Training uses batches of 128 with shuffling and `drop_last`, giving 390 iterations per epoch; class labels are not used. Fig. 1 shows ten images from the first training batch.

TABLE I
Model Architecture and Training Configuration

Item	Value (from notebook)
Input / data	CIFAR-10, 32×32 RGB, range $[-1, 1]$
Patch sizes	4 (64 tokens), 8 (16 tokens)
Embedding dim. / depth / heads	128 / 4 layers / 4
Feed-forward width	512 (GELU)
Time conditioning	Sinusoidal $\rightarrow$ MLP, added to all tokens
Parameters (Patch-4)	846,768; Patch-8 and no-time: N/A
Optimizer	AdamW, lr 10^{-4}, weight decay 0.01
Epochs / batch size	10 / 128 (390 iterations per epoch)
Gradient clipping	Max-norm 1.0 (main models only)
Time sampling	$t = \sigma(0.8n - 0.8)$, $n \sim \mathcal{N}(0, 1)$
Time clipping	$t \in [0.001, 0.995]$
Sampler	40 Euler steps, $t : 0 \rightarrow 1$
Seed / hardware	42 / Tesla T4 (CUDA)

III. Model Architecture

The model (`SmallJiT`) follows a ViT-style design built on the Transformer encoder [3]. Each $3 \times 32 \times 32$ image is split into non-overlapping $p \times p$ patches ($p \in \{4, 8\}$), giving 64 tokens for p=4 and 16 tokens for p=8. Each flattened patch (dimension $3p^2$) is linearly projected to $d = 128$ dimensions and a learned positional embedding is added. The time t is encoded with sinusoidal features (frequencies decaying geometrically from 1 to 10^{-4}, base 10000), passed through a two-layer SiLU MLP, and added to every token. Four post-norm Transformer encoder layers (4 heads, GELU, feed-forward width 512, PyTorch default dropout) process the tokens, and a linear layer maps each token back to $3p^2$ pixel values, which are reassembled into the image. The Patch-4 model has 846,768 parameters (printed in the notebook). Table I lists the configuration.

IV. Diffusion Formulation and Loss

Let $\mathbf{x}$ be a clean image and $\varepsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ Gaussian noise. For a time $t \in (0, 1)$ the noisy input is the linear interpolation

$$\mathbf{z}_t = t\,\mathbf{x} + (1 - t)\,\varepsilon, \tag{1}$$

so $t = 0$ is pure noise and $t = 1$ is the clean image. The network predicts the clean image,

$$\hat{\mathbf{x}} = f_\theta(\mathbf{z}_t, t), \tag{2}$$

where f_θ is the Transformer with parameters θ. The prediction is converted to a velocity estimate and compared with the target velocity:

$$\hat{\mathbf{v}} = \frac{\hat{\mathbf{x}} - \mathbf{z}_t}{\max(1-t, 0.05)}, \tag{3}$$

$$\mathbf{v} = \frac{\mathbf{x} - \mathbf{z}_t}{\max(1-t, 0.05)}, \tag{4}$$

$$\mathscr{L} = \mathbb{E}_{\mathbf{x},t,\varepsilon}\left[\|\hat{\mathbf{v}} - \mathbf{v}\|^2\right]. \tag{5}$$

Here $\mathbf{v}$ is the target velocity, $\hat{\mathbf{v}}$ its estimate, and the $\max(\cdot, 0.05)$ term prevents division by values near zero as $t \to 1$. In the notebook (`diffusion_training_loss`) the loss is `F.mse_loss(v_pred, v_target)`, i.e. the squared error *averaged over all pixels* rather than summed; Eq. (5) is therefore the per-element mean. The equations above match the code line by line, and the loss was not altered. Because $\hat{\mathbf{v}} - \mathbf{v} = (\hat{\mathbf{x}} - \mathbf{x})/\max(1-t, 0.05)$, the objective is an $\mathbf{x}$-prediction error weighted by $1/\max(1-t, 0.05)^2$, which emphasizes accuracy at large t. Time is sampled from a logit-normal-style distribution ($\mu = -0.8$, $\sigma = 0.8$), which favours noisier inputs ($t < 0.5$) as in the spirit of [1].

Sampling. Starting from $\mathbf{z}_0 \sim \mathscr{N}(\mathbf{0}, \mathbf{I})$ on a grid $0 = t_0 < \cdots < t_{40} = 1$, the sampler applies $\mathbf{z}_{i+1} = \mathbf{z}_i + (t_{i+1} - t_i)\,\hat{\mathbf{v}}(\mathbf{z}_i, t_i)$ and clamps the final output to $[-1, 1]$.

V. Experimental Setup

Both time-conditioned models (Patch-4, Patch-8) are trained for 10 epochs with the settings of Table I. The ablation model (`SmallJiTNoTime`) is identical to Patch-4 except that the time embedding is removed; it receives t in the loss but ignores it. Training loss is the mean over the 390 batches of each epoch. Test loss is the same stochastic v-loss (random t and ε) averaged over the full 10,000-image test set in evaluation mode. It is therefore a noise-weighted denoising error, *not* a generative-quality metric such as FID, and was computed once per model. One implementation detail matters for interpretation: the notebook's ablation training loop does not apply the gradient clipping used for the two main models.

VI. Results and Analysis

Fig. 2 shows the training curves and Table II the final values. Both models start high (0.358753 and 0.432867 in epoch 1) and decrease steadily, reaching 0.211430 and 0.206700 after 10 epochs. The curves cross around epoch 4: Patch-8 starts higher but ends lower. The test v-loss values (0.204914 and 0.201217) lie slightly below the final training losses. This is plausible because the training value is an average over a whole epoch while the model improves, and because both are noisy single estimates; it does not indicate over-fitting. Patch-8 has the lower test v-loss by 0.003697 (1.8% of the Patch-4 value). This is a small gap from one run per model, so the evidence does not establish that Patch-8 is generally better. Since the loss is still falling at epoch 10, all models are under-trained.

VII. Ablation Study: Effect of Time Embedding

To isolate time conditioning, Patch-4 with time embedding is compared with Patch-4 without it; everything else is held fixed except the clipping detail noted in Section V. This differs from the patch-size comparison (Patch-4 vs. Patch-8) discussed above. Table III gives the comparison and Fig. 3 the notebook's ablation graph. That graph is a bar chart of test v-loss for all three runs; the time-embedding effect is the difference between its first and third bars, while the middle bar is the patch-size result.

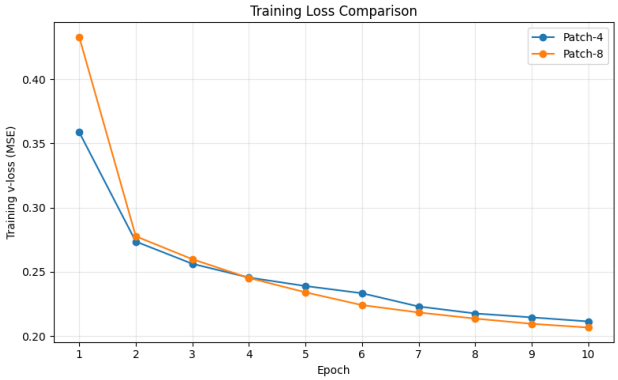


Fig. 2. Training v-loss (MSE) per epoch for Patch-4 and Patch-8 over 10 epochs, from the notebook.

TABLE II
Final Experimental Results (v-loss, MSE)

Configuration	Initial train	Final train	Test
Patch-4 (with time emb.)	0.358753	0.211430	0.204914
Patch-8 (with time emb.)	0.432867	0.206700	0.201217
Patch-4 (no time emb.)	0.3765*	0.216322	0.208269

*Printed to four decimals in the notebook (epoch-1 loss 0.3765); the six-decimal value is N/A. Lower is better.

Removing time embedding raises the Patch-4 test v-loss from 0.204914 to 0.208269, an absolute increase of 0.003355 (about 1.64%). The direction agrees with a technical expectation: the target in Eq. (4) is weighted by $1/\max(1-t, 0.05)$, and the optimal clean-image prediction depends on how much signal $\mathbf{z}_t$ carries. Without t, the network must infer the noise level from the input alone, which is possible but harder. The small size of the gap also fits the setting, since the noise level can be partly estimated from the pixel statistics of $\mathbf{z}_t$. *Limitations:* there is one run per configuration, no repeated seeds and no confidence intervals. Test losses are themselves stochastic, and the ablation loop lacked gradient clipping. The result should therefore be read as an observation consistent with a benefit from time conditioning, not as a proven causal effect.

VIII. Comparison with the Original Paper

Table IV contrasts the original JiT paper [1] with this project. Properties of [1] are limited to what its abstract states; entries marked N/A are not reported in this document.

Similarities. Both use a Transformer over pixel patches, avoid a tokenizer and pre-training, and predict the clean image rather than noise. *Differences.* The project is orders of magnitude smaller, uses a much lower resolution and smaller patches, trains for a few minutes, and omits the paper's evaluation protocol. The paper's reported results come from [1] and are not reproduced here. Test v-loss measures denoising error under a time-weighted objective and *must not* be compared numerically with FID or Inception Score. This project therefore does not

TABLE III
ABLATION RESULTS: PATCH-4 WITH VS. WITHOUT TIME EMBEDDING

Model	Final train	Test
Patch-4, with time embedding	0.211430	0.204914
Patch-4, without time embedding	0.216322	0.208269
Absolute difference	0.004892	**0.003355**

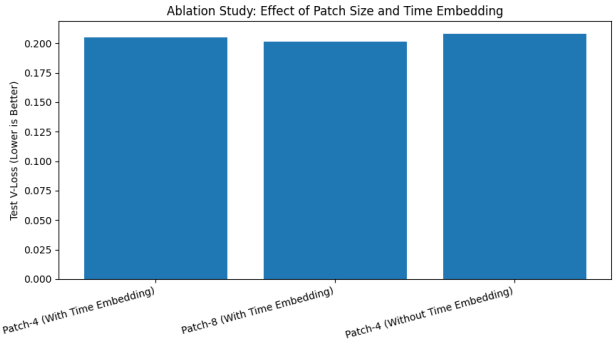


Fig. 3. Notebook ablation graph: test v-loss (lower is better) for Patch-4 with time embedding, Patch-8 with time embedding, and Patch-4 without time embedding. The y-axis starts at zero, so the bars look similar although the differences are in the third decimal place.

claim to reproduce, match, or approximate the paper's quality; it only illustrates the clean-image-prediction idea at small scale.

IX. GENERATED SAMPLES AND DENOISING ANALYSIS

Fig. 4 shows eight 40-step samples from each model. The images exhibit global colour and brightness layout (sky-like or dark regions, a green-brown scene) but no recognizable objects. Patch-4 samples show a visible 4×4 grid structure, and Patch-8 samples show coarser 8×8 blocks with smoother content inside each block. The seams between patches are a clear artifact of independent per-patch outputs in a small, briefly trained model. Samples are drawn once and not selected, and no quantitative sample metric was computed.

Figs. 5 and 6 show denoising at a single time $t = 0.5$ on the first five test images: clean images (top), the noisy input $\mathbf{z}_{0.5} = 0.5\mathbf{x} + 0.5\varepsilon$ (middle), and the one-step prediction $\hat{\mathbf{x}}$ (bottom). The noisy inputs are nearly unrecognizable, yet both models recover the global colour layout, for example the green frog scene, the bright sky and dark hull of the boats, and the pale background of the sign in the fourth image. Fine detail such as edges, texture and object contours is lost, and the outputs are blocky and blurry, as expected for a regression (MSE-type) objective. Patch-4 preserves somewhat more spatial variation, while Patch-8 yields coarser, smoother blocks; these are visual impressions only and do not by themselves explain the small test-loss difference.

X. LIMITATIONS AND FAILURE CASES

- **Scale:** a 0.85M-parameter model, 32×32 resolution, and 10 epochs are far from the paper's regime.
- **Single run:** no seeds are repeated, so all differences (0.003697 between patch sizes, 0.003355 for time embedding) are within what run-to-run variation could plausibly produce; no variance is reported.
- **Stochastic metric:** test v-loss depends on random t and ε and was evaluated once; it does not measure sample quality.

TABLE IV
ORIGINAL JIT PAPER [1] VS. THIS PROJECT

Aspect	Original paper [1]	This project
Dataset	ImageNet	CIFAR-10
Resolution	256 and 512	32×32
Architecture	Plain large-patch Transformer on raw pixels; no tokenizer, pre-training, or extra loss	4-layer, d=128 Transformer on raw pixels; no tokenizer or pre-training
Patch size	16 and 32	4 and 8
Model scale	Multiple sizes; see [1]	0.85M parameters (Patch-4)
Prediction	Clean image ($\mathbf{x}$-prediction)	Clean image $\hat{\mathbf{x}}$, loss in $\mathbf{v}$-space
Training setup	See [1]	10 epochs, batch 128, one T4 GPU, about 3 min per model
Metrics	Generative metrics reported in [1]	Test v-loss only; no FID or IS
Objective	Competitive generation at scale	Small-scale implementation and ablation



Fig. 4. Generated CIFAR-10 samples (40 Euler steps). Top row: Patch-4 model; bottom row: Patch-8 model.

- **No generative metric:** FID and Inception Score were not computed, so claims about sample quality are qualitative only.
- **Ablation confound:** the no-time model was trained without gradient clipping; the effect of time embedding is therefore not perfectly isolated.
- **Visible artifacts:** blocky patch seams, blur, and no recognizable objects in generated samples (Fig. 4).

XI. CONCLUSION AND FUTURE WORK

The notebook implements an end-to-end JiT-inspired pipeline: CIFAR-10 loading, a patch-based Transformer that predicts clean images, v-loss training, test evaluation, Euler sampling, denoising visualization, and an ablation. Patch-8 reached the lowest test v-loss (0.201217), and removing time embedding raised the Patch-4 test v-loss by 0.003355. These are observations from single runs of a small experiment, not evidence of state-of-the-art generation. Future work includes longer training, multiple seeds with mean and standard deviation, re-running the ablation with identical clipping, EMA and learning-rate scheduling, larger models, and FID evaluation of generated samples.

REFERENCES

[1] T. Li and K. He, "Back to basics: Let denoising generative models denoise," arXiv:2511.13720v2, 2026.

[2] A. Krizhevsky, "Learning multiple layers of features from tiny images," Univ. Toronto, Tech. Rep., 2009.

[3] A. Vaswani *et al.*, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017.



Fig. 5. Patch-4 denoising at $t = 0.5$ (notebook output). Rows: clean test image, noisy input $\mathbf{z}_t$, and predicted clean image $\hat{\mathbf{x}}$.

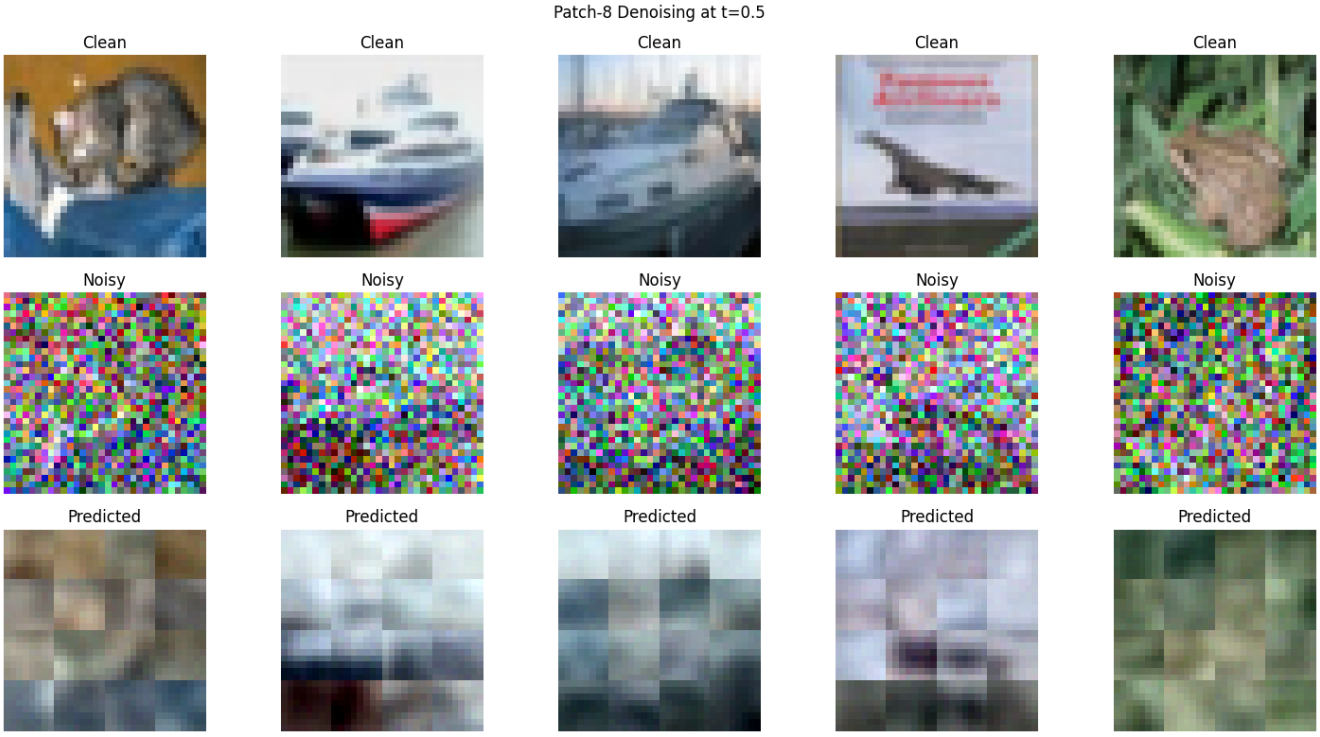


Fig. 6. Patch-8 denoising at $t = 0.5$ (notebook output). Rows are as in Fig. 5.